

Tipos *de dados*

Todo valor em Python tem um **tipo**: texto, número inteiro, número decimal, verdadeiro ou falso, "nada". O tipo decide o que dá para fazer com o valor. Nesta aula: variáveis, os cinco tipos básicos, `type()`, as surpresas do `float`, conversões entre tipos e o que acontece quando uma conversão falha.

Variáveis

`str` · `int` · `float` · `bool` · `None`

`type()`

`int()` · `float()` · `str()` · `bool()`

`isinstance()`

`ValueError`

Formação Claude Code 2026 · Baseado em docs.python.org (Built-in Types, Floating-Point Arithmetic, Built-in Functions), consultados em outubro de 2026 · Exemplos testados com Python 3.14 na pasta `exemplos/` · Saídas reais

Variáveis: *nomes para valores*

Uma **variável** é um nome que aponta para um valor. Você cria com `=` (atribuição) e pode trocar o valor depois. Em Python não se declara o tipo: ele vem do valor.

```
idade = 29          # cria a variável
idade = idade + 1    # usa e troca o valor
print(idade)         # 30
```

O `=` não é "igual" da matemática: é "guarde este valor neste nome". Para **comparar**, usa-se `==` (aula 06).

REGRA DE NOME	EXEMPLO
Letras, números e _	<code>nota_final</code> , <code>aluno2</code>
Não começa com número	<code>2aluno</code> é erro
Sem espaço nem hífen	<code>nota-final</code> é erro
Maiúscula ≠ minúscula	<code>Idade</code> e <code>idade</code> são diferentes
Não use palavras do Python	<code>if</code> , <code>for</code> , <code>class</code> ...

Convenção do Python (PEP 8): **snake_case**, minúsculas com `_`. Nomes que dizem o que guardam: `preco_total` , não `x` .

Os cinco *tipos básicos*

tipos.py

```
nome = "Ana"           # str: texto
idade = 29             # int: número inteiro
altura = 1.68          # float: número com casas decimais
estudante = True       # bool: verdadeiro ou falso
apelido = None         # NoneType: "nenhum valor"

print(type(nome))
print(type(idade))
print(type(altura))
print(type(estudante))
print(type(apelido))
```

SAÍDA

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'NoneType'>
```

TIPO	GUARDA
str	Texto, entre aspas
int	Inteiros: -3, 0, 42
float	Decimais, com ponto : 1.68
bool	<code>True</code> ou <code>False</code> (com maiúscula)
NoneType	<code>None</code> : ausência de valor

int, float *e suas surpresas*

numeros.py

```
# int não tem limite de tamanho
print(2 ** 100)

# float é aproximado: alguns decimais não são exatos
print(0.1 + 0.2)
print(round(0.1 + 0.2, 2))

# Divisão com / sempre dá float, mesmo quando é exata
print(10 / 2)
print(type(10 / 2))

# Separador de milhar com _ (só para leitura)
populacao = 203_080_756
print(populacao)
```

SAÍDA

```
1267650600228229401496703205376
0.30000000000000004
0.3
5.0
<class 'float'>
203080756
```

- ✓ `int` não estoura: `2 ** 100` sai inteiro
- ✓ `0.1 + 0.2` dá `0.30000000000000004`: o `float` é guardado em binário e alguns decimais não são exatos
- ✓ Para dinheiro, arredonde ao mostrar (`round` , ou `:.2f` na aula 05)
- ✓ `/` sempre devolve `float`

Não é bug do Python: quase todas as linguagens se comportam assim com ponto flutuante. Contas financeiras exatas usam o módulo `decimal` .

Convertendo *entre tipos*

conversoes.py

```
print(int("42") + 1)      # texto -> inteiro
print(float("3.5") * 2)   # texto -> float
print(str(10) + "0")      # inteiro -> texto
print(int(9.99))          # float -> int: corta as decimais
print(round(9.99))        # para arredondar, use round()

# bool(): vazio e zero são False; o resto é True
print(bool(0), bool(""), bool(None))
print(bool(7), bool("oi"), bool(" "))
```

SAÍDA

```
43
7.0
100
9
10
False False False
True True True
```

FUNÇÃO	CONVERTE PARA
int()	Inteiro (corta decimais)
float()	Decimal
str()	Texto
bool()	True ou False
round()	Arredonda

Quando a conversão *falha*

erro_conversao.py

```
idade = int("vinte")
```

```
Traceback (most recent call last):  
  File "erro_conversao.py", line 1, in <module>  
    idade = int("vinte")  
ValueError: invalid literal for int() with base 10: 'vinte'
```

- ✓ **ValueError**: o tipo está certo (é um texto), mas o **valor** não serve
- ✓ `int("42")` funciona; `int("vinte")` e `int("4.5")` não
- ✓ Para `"4.5"`, converta com `float()`

Esse erro vai aparecer muito na próxima aula: tudo o que vem do teclado é texto, e o usuário pode digitar qualquer coisa. Tratar o erro é assunto do capítulo 12 do guia.

type() e isinstance()

instance_ex.py

```
valor = 3.14
print(isinstance(valor, float))      # True
print(isinstance(valor, int))        # False
print(isinstance(valor, (int, float))) # True: é um dos dois
```

SAÍDA

```
True
False
True
```

- ✓ `type(x)` mostra o tipo: ótimo para investigar no REPL
- ✓ `isinstance(x, tipo)` responde `True` ou `False`: melhor para decisões no código
- ✓ Aceita uma tupla de tipos: "é int ou float?"

Detalhe curioso: `bool` é um tipo especial de `int`. `True` vale 1 e `False` vale 0, por isso `True + 1` dá 2.

Tipos que vêm *nas próximas aulas*

TIPO	EXEMPLO	MUTÁVEL?	AULA
str	"Python"	Não	07 e 08 · Strings
list	[1, 2, 3]	Sim	09 · Listas
tuple	(1, 2, 3)	Não	10 · Tuplas
set	{1, 2, 3}	Sim	11 · Set
dict	{"nome": "Ana"}	Sim	12 · Dicionário

Mutável = dá para alterar o valor sem criar outro. Essa ideia separa listas de tuplas e explica vários comportamentos que aparecem nas aulas 07 a 10.

Investigando tipos *com o Cursor*

Sou iniciante em Python. Por que `0.1 + 0.2` não dá `0.3` exatamente? Explique com uma analogia simples e diga como devo mostrar valores em dinheiro.

Faça 5 perguntas de "qual é o tipo deste valor?" para eu responder, com valores como `"10"`, `10`, `10.0` e `True`. Só me diga as respostas depois que eu tentar.

- ✓ Peça **analogias** e exemplos curtos
- ✓ Peça **exercícios** no seu nível, não respostas prontas
- ✓ Confira cada resposta no REPL: `type(10.0)`

O REPL é o seu laboratório: qualquer dúvida de tipo se resolve em um segundo com `type()`.

Descubra *e converta*

Enunciado: numa lista com `"42"`, `3.7`, `True` e `"2.5"`, mostre o tipo dos três primeiros; depois dobre o `"42"` como número, converta `3.7` em inteiro, some 1 a `"2.5"` e a `True`.

Uma resposta possível

```
# Exercício: descubra o tipo e converta
valores = ["42", 3.7, True, "2.5"]
print(type(valores[0]), type(valores[1]), type(valores[2]))
print(int(valores[0]) * 2)      # "42" -> 42
print(int(valores[1]))         # 3.7 -> 3
print(float(valores[3]) + 1)   # "2.5" -> 3.5
print(int(valores[2]) + 1)     # True vale 1
```

SAÍDA

```
<class 'str'> <class 'float'> <class 'bool'>
84
3
3.5
2
```

`valores[0]` pega o primeiro item da lista. Listas são o assunto da aula 09; por ora, basta saber que a posição começa em 0.

Teste o que *ficou*

1. Qual é o tipo de `10 / 2`?

A) int

B) float

C) str

D) bool

B. A divisão com `/` sempre dá float (5.0).

2. O que `int(9.99)` devolve?

A) 10

B) 9

C) 9.99

D) Erro

B. Corta as decimais; para arredondar, `round()`.

3. `int("vinte")` gera qual erro?

A) `TypeError`

B) `NameError`

C) `ValueError`

D) Nenhum

C. O valor não é um número válido.

4. Qual destes é False?

A) `bool(" ")`

B) `bool("0")`

C) `bool(0)`

D) `bool(-1)`

C. `" "` e `"0"` são textos não vazios.

Cada valor *tem um tipo*

Variáveis guardam valores, e o tipo vem do valor: `str`, `int`, `float`, `bool` e `None`.
`type()` e `isinstance()` investigam; `int()`, `float()`, `str()` e `bool()` convertem; e
um valor que não serve gera **ValueError**.

snake_case

str · int · float · bool · None

0.1 + 0.2

Conversões

ValueError

Próxima aula → 04 · Utilizando o Input

Fontes: docs.python.org/3/library/stdtypes.html; docs.python.org/3/tutorial/floatingpoint.html; docs.python.org/3/library/functions.html; peps.python.org/pep-0008, consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.